ADAGDS — Complete Project Documentation

Adaptive Distributed Application Generation & Deployment System

COMSATS University Islamabad, Lahore Campus Course: Parallel and Distributed Computing (CSC 334) — Final Lab Exam, Spring 2026 CLO-5: *Develop an application using concepts of distributed and parallel computing.*

Table of Contents

1. Introduction & Problem Statement
2. Technology Stack
3. Database & Persistence Model
4. High-Level Architecture
5. Execution Lifecycle (the Pipeline)
6. Master–Worker Communication Protocol
7. Complete File-by-File Reference
8. The Generated Application
9. Requirement-to-Implementation Map
10. Fault Tolerance — In Detail
11. How to Run & Use
12. Testing
13. Verification Evidence

---

1. Introduction & Problem Statement

Modern distributed systems must adapt to the resources actually available before they run large workloads. **ADAGDS** is a system that:

- **analyses** real-time machine resources (CPU, memory, optional GPU, workers),
- **adaptively decides** how many applications to generate (between 1 and 20),
- **distributes** the build jobs across a master/worker cluster,
- **generates** each application as a *fully independent project codebase*,
- **deploys and launches** the applications, each on a unique endpoint,
- **monitors** the entire run on a live web dashboard,
- and does all of this with **fault tolerance** — surviving worker failure without losing a single job.

The system is driven by a single shared codebase and a set of JSON configuration files, each representing one *tenant* (one application specification).

---

2. Technology Stack

Layer	Choice	Why
Programming language	**JavaScript**	Single language across master, workers, generated apps and dashboard; native support for asynchronous, event-driven distributed code.
Runtime	**Node.js 18+** (CommonJS modules)	Built-in `child_process`, `http`, `os`, `events` — everything needed for a multi-process distributed system with no heavy framework.
HTTP / web server	**Express 4**	Lightweight server for the master API, the worker nodes, and every generated application.
Real-time channel	**ws** (WebSocket)	Pushes live state and logs to the dashboard with no polling.
Reverse proxy	**http-proxy-middleware 2**	Routes `http://localhost:8080/appN` to each deployed application.
Resource probing	**systeminformation**	Cross-platform CPU-load and optional GPU detection (`os.loadavg()` is unreliable on Windows).
Testing	**node:test** (built into Node)	Zero extra dependencies; tests live in `/tests` mirroring `/src`.
Dashboard UI	**HTML + CSS + vanilla JavaScript**	No build step, no framework — instantly viewable.

Production dependencies (4 total): `express`, `ws`, `http-proxy-middleware`, `systeminformation`. Each **generated** application depends only on `express`.

The codebase follows KISS, DRY and SOLID: one responsibility per module, named constants (no magic numbers), explicit error handling, and modular boundaries.

---

3. Database & Persistence Model

ADAGDS does not use a traditional database. There is no SQL or NoSQL server (no MySQL, PostgreSQL, MongoDB, etc.). This is a deliberate design decision — a build-orchestration system has no relational/transactional data; introducing a database server would violate KISS and add an unnecessary moving part.

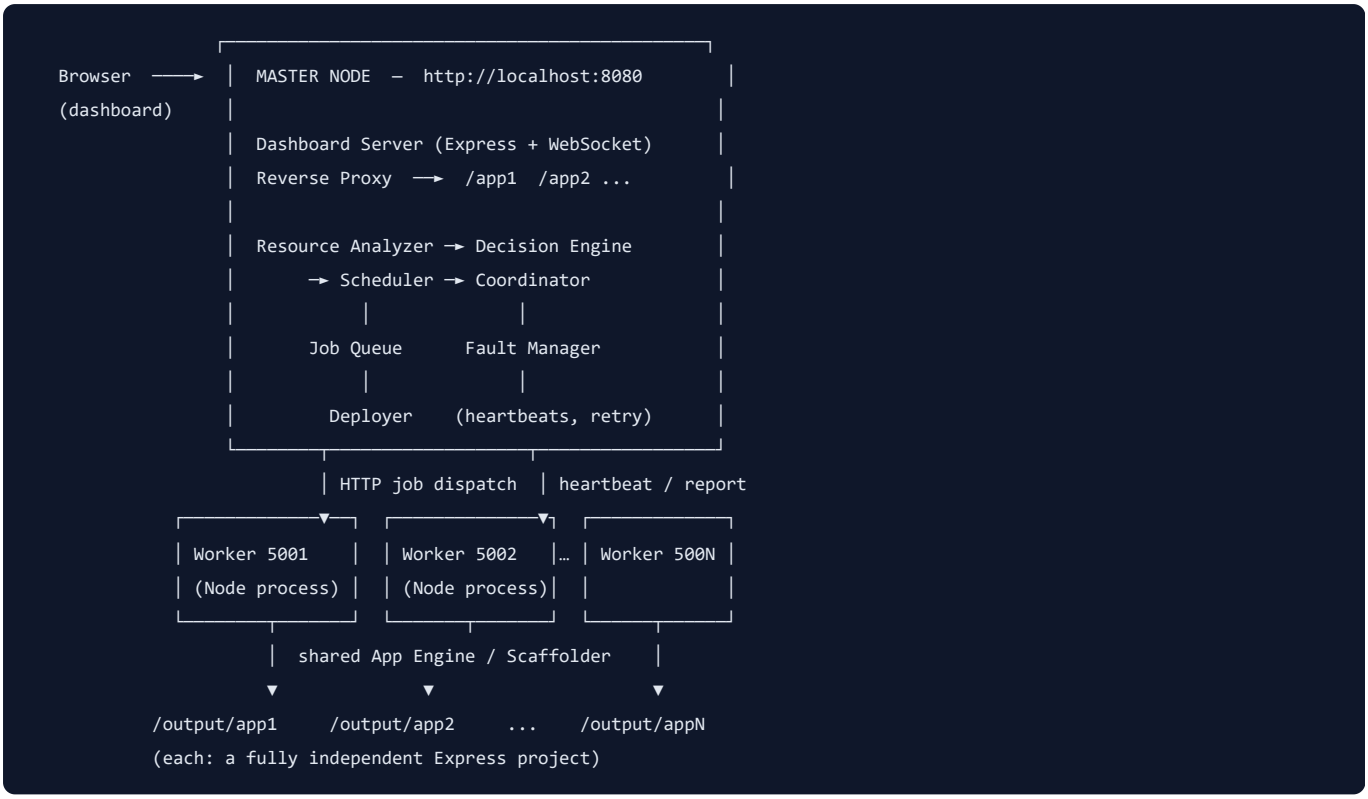
Instead, persistence is **file-based JSON**, which is the natural fit:

Data	Location	Role
Tenant specifications	`config/tenants/*.json`	**Input** — 15 application specs (the "10–20 JSON configs").
System tunables	`config/system.json`	Ports, limits, low-resource simulation toggle.
Job-queue state	`logs/queue-state.json`	**Runtime persistence** — every job state change is flushed to disk so an interrupted run is never lost (Requirement 11).
Structured logs	`logs/master.log`, `logs/worker-*.log`	Append-only JSON-lines log of every event and decision.
Per-app config	`output/appN/config/app.config.json`	Each generated app's **own isolated data store** (Requirement 6).
Uploaded dataset	`config/data/active-dataset.json`	A dataset uploaded for split-across-N generation — sliced into per-app files, still just JSON files.
Per-app data slice	`output/appN/data/records.json`	A data app's own slice of the dataset, served directly from disk.

Every generated application reads its configuration from its **own** `config/app.config.json` file — it has its own isolated configuration space and shares no runtime state with any other app. The same is true of uploaded datasets: a 20 MB file like a Quran-verses dataset is handled entirely as plain JSON files — **no database is ever involved.**

---

4. High-Level Architecture



- The **master** is one Node process. It serves the dashboard, runs the decision pipeline, and coordinates everything.
- Each **worker** is a *separate* Node process (a separate "node"), reachable over HTTP. They communicate with the master via JSON HTTP calls.
- The **generated apps** land in `/output` and are launched as their own independent processes.

---

5. Execution Lifecycle (the Pipeline)

When the operator clicks **Start Generation**, the coordinator runs six phases:

1. **ANALYZING** — the Resource Analyzer captures a snapshot: CPU cores, CPU load %, free/total memory, optional GPU, and the count of healthy workers.
2. **Decide** — the Decision Engine computes resource ceilings and picks the number of apps to generate (1–20). The Scheduler computes the optimal parallel-execution level.
3. **GENERATING** — one `Job` is created per app and added to the Job Queue. The coordinator dispatches jobs to idle workers, up to the parallelism limit. Workers build apps concurrently and report results.
4. **MINIMUM_GUARANTEE** — if fewer than 2 apps succeeded, the master builds the remainder itself, in-process, guaranteeing at least 1–2 working apps.
5. **DEPLOYING** — the Deployer installs dependencies and launches every generated app on its own unique port; the reverse proxy exposes each at `/appN` so all of them get a clickable link.
6. **DONE** — final statistics are logged; the dashboard shows everything.

Throughout, every event and **system decision** is logged and streamed live to the dashboard over WebSocket.

---

6. Master–Worker Communication Protocol

All inter-node messages are JSON over HTTP, each with an enforced timeout.

Endpoints served by the MASTER (port 8080):

Method & path	Caller	Purpose
`POST /api/worker/register`	worker	Worker announces it is listening.
`POST /api/worker/heartbeat`	worker	Periodic "I am alive" + stats.
`POST /api/worker/report`	worker	Reports a build success/failure.
`POST /api/start`	dashboard	Triggers the pipeline (optional `appCount`).
`POST /api/upload-tenant`	dashboard	Uploads and registers a new tenant config.
`POST /api/upload-data`	dashboard	Uploads a dataset to split across apps.
`POST /api/clear-dataset`	dashboard	Discards the active dataset.
`GET /api/state`	dashboard	Returns the full system state.
`GET /api/analyze`	dashboard	Captures a fresh resource snapshot.
`POST /api/kill-worker`	dashboard	Fault simulation — kills a worker.
`GET /appN/*`	browser	Reverse-proxied to a deployed app.

Endpoints served by each WORKER (ports 5001+):

Method & path	Purpose
`POST /job`	Accept a build job (acknowledged immediately, built async).
`GET /health`	Liveness/stats probe.
`POST /shutdown`	Graceful shutdown.

---

7. Complete File-by-File Reference

Root files

`package.json` — Project manifest. Declares the 4 production dependencies, the Node 18+ engine requirement, and three scripts: `start` (boots the master), `start:worker` (runs a standalone worker), and `test` (runs the unit tests).

`package-lock.json` — Auto-generated dependency lock file ensuring reproducible installs.

`.gitignore` — Excludes `node_modules/`, `output/`, `logs/`, `.env` and OS junk from version control.

`README.md` — Quick-start guide: setup, run, requirement map, and the demo instructions for fault tolerance and low-resource mode.

`DOCUMENTATION.md` — This document.

`1- Statement.pdf` — The original exam question paper.

`config/` — Configuration (the system's input data)

`config/system.json` — System tunables: `appCount` (set a number to generate exactly that many apps, or `"auto"` to let the system decide), the `forceLowResources` toggle and its `simulatedLowResources` values, filesystem paths, worker count (`"auto"`), and deployment options (`autoLaunchCount`, `runNpmInstall`).

`config/tenants/tenant-01.json` … `tenant-15.json` — 15 **tenant specifications**. Each is one application's definition and contains: `id`, `name`, `slug`, `version`, `features` (feature flags), `environment` (stage, region, log level), `dataSource` (type + endpoint abstraction), and `theme` (colors, layout, font, logo). The 15 tenants:

File	Application	Layout	Data source
tenant-01	Aurora Analytics	sidebar	rest
tenant-02	Nimbus CRM	topnav	graphql
tenant-03	Quartz Commerce	grid	rest
tenant-04	Helios Dashboard	minimal	mock
tenant-05	Vertex Tasks	sidebar	rest
tenant-06	Coral Booking	topnav	rest
tenant-07	Slate Docs	minimal	file
tenant-08	Pulse Monitor	grid	websocket
tenant-09	Onyx Finance	sidebar	rest
tenant-10	Lumen Learning	topnav	graphql
tenant-11	Forge DevOps	grid	rest
tenant-12	Tidal Media	topnav	rest
tenant-13	Ember Support	sidebar	mock
tenant-14	Mint Health	grid	rest
tenant-15	Cobalt Travel	topnav	graphql

`src/` — The shared base codebase

`src/index.js` — The application entrypoint. A two-line file that boots the master node by requiring `master/master.js`.

`src/constants.js` — The single source of truth for every named constant: network ports (master 8080, workers 5001+, apps 4001+), worker-pool bounds, the adaptive-decision bounds (`MIN_APPS` 1, `MAX_APPS` 20), the decision cost model (`PER_APP_MEM_GB`, `CPU_APPS_PER_CORE`, `APPS_PER_WORKER`, `HIGH_LOAD_PCT`), fault-tolerance settings (heartbeat intervals, `MAX_RETRIES`, `GUARANTEED_MIN_BUILDS`), and the frozen enums `JOB_STATE`, `WORKER_STATE`, `PHASE`, and `LOG_LEVEL`.

`src/common/` — Shared utilities

`src/common/logger.js` — A structured, event-emitting logger (a singleton class extending `EventEmitter`). Every entry is written to the console *and* a per-process log file, and emitted as a `log` event so the dashboard can stream it. Provides `debug/info/warn/error` plus a special **`decision()`** level used to highlight system decisions for grading (Requirement 12).

`src/common/config-loader.js` — Loads and **validates** JSON configuration. `validateTenant()` checks every required field, the kebab-case slug, the theme and data-source sub-objects, throwing descriptive errors. `loadTenants()` reads the whole tenants directory, rejects invalid JSON and duplicate IDs, and returns a sorted array. `loadSystemConfig()` loads `system.json`.

`src/common/messaging.js` — `requestJson()`, a minimal JSON-over-HTTP client used for *all* master↔worker traffic. Every request has an enforced timeout so a stalled node can never hang the coordinator. Rejects on network error, timeout, or non-2xx status.

`src/common/chunker.js` — `splitIntoChunks()`, a pure function that divides a dataset into N evenly-sized contiguous slices. It is the core of the "split a dataset across N apps" feature, and is fully unit-tested.

`src/resource/` — Resource awareness and decision-making

`src/resource/resource-analyzer.js` — Produces a resource snapshot. `analyze()` reads CPU cores and memory from Node's `os` module and CPU load + GPU from `systeminformation`. If `forceLowResources` is set in `system.json`, it returns a deterministic simulated low-resource snapshot (used for the demo). GPU detection is optional — its absence is never an error.

`src/resource/decision-engine.js` — The **Adaptive Workload Decision Engine** (Requirement 8). The pure function `decideAppCount()` takes the demand — either an explicit `appCount` from `system.json`, or the number of tenant configs in auto mode — together with three resource ceilings (by memory, by CPU, by workers). It picks the smallest (the *binding constraint*), halves it if CPU load is high, and clamps the result to [1, 20]. It returns the number plus human-readable `reasoning`, logged at `decision` level. Being a pure function, it is fully unit-testable.

`src/resource/scheduler.js` — The **Resource-Aware Scheduler** (Requirement 7). `computeParallelism()` decides how many builds may run at once, bounded by the work available, the live worker pool, and (cores − 1) so the master keeps a core. `pickIdleWorker()` selects the next free worker.

`src/engine/` — The shared generation engine (Requirement 1 & 5)

`src/engine/app-engine.js` — The shared **Application Generation Engine**. `buildApp()` is what every worker calls: it generates one project via the scaffolder, verifies it, times it, and returns a structured build result. `buildDataApp()` does the same for a **data app** — one slice of an uploaded dataset.

`src/engine/scaffolder.js` — The heart of **Requirement 5**. `generate()` writes a *fully independent project* for one tenant. It renders templates with `{{PLACEHOLDER}}` substitution, writes to a `.tmp` directory first, then **atomically renames** it into place — so a crash mid-write never leaves a half-built project (partial-build recovery). `generateDataApp()` builds a data app and bundles its slice of the dataset as `data/records.json`. `verify()` confirms every required file exists; `portForIndex()` assigns each app a unique port.

`src/engine/templates/` — The 9 base template files used to build each app:

Template	Becomes	Contents
`package.json.tpl`	`package.json`	App manifest; only dependency is `express`.
`server.js.tpl`	`src/server.js`	Express server: a tiny dependency-free `.env` loader, static hosting, `/api/info`, `/health`.
`config.js.tpl`	`src/config.js`	Loads the app's own `config/app.config.json`.
`index.html.tpl`	`public/index.html`	Themed, configuration-driven page.
`styles.css.tpl`	`public/styles.css`	Theme colors/fonts injected from the tenant.
`app.js.tpl`	`public/app.js`	Front-end: fetches `/api/info` and renders flags.
`build.js.tpl`	`build.js`	Build-validation script for the app.
`env.tpl`	`.env`	Environment variables (PORT, APP_ID, ...).
`readme.tpl`	`README.md`	Per-app run instructions.

Four further templates — `data-server.js.tpl`, `data-index.html.tpl`, `data-styles.css.tpl`, `data-app.js.tpl` — are used to build **data apps**: each serves and displays its own slice of an uploaded dataset (see *Dataset Mode*).

`src/jobs/` — Job model and queue

`src/jobs/job.js` — The `Job` class: one build job for one tenant. It holds the tenant, app index, state, attempt count, assigned worker, result/error, and timing. Its state-machine methods (`markAssigned`, `markInProgress`, `markCompleted`, `markFailed`, `resetForRetry`) drive the lifecycle.

`src/jobs/job-queue.js` — The `JobQueue` class: a FIFO queue with full lifecycle tracking. `nextPending()`, `byState()`, `isDrained()` and `stats()` support the coordinator. Critically, **every change is flushed to `logs/queue-state.json`** via `commit()` — this is the "no job loss" guarantee.

`src/worker/` — The worker node

`src/worker/worker.js` — A complete, *independent* worker process. On boot it starts an Express server, registers with the master, and begins sending heartbeats. On `POST /job` it acknowledges immediately, runs the shared `app-engine.buildApp()`, then reports success or failure back to the master. Configuration (its ID, port, the master address) arrives via environment variables, so it can also be launched as a real remote node.

`src/master/` — The master node

`src/master/master.js` — The bootstrap. It configures logging, loads the configs, creates the Fault Manager, Coordinator and Dashboard Server, **spawns the worker child processes**, installs graceful-shutdown handlers, and captures an initial resource snapshot.

`src/master/coordinator.js` — The orchestration brain. `runPipeline()` runs all six phases. It creates jobs, dispatches them to workers in parallel (`_dispatchToWorkers`), handles worker reports (`handleWorkerReport`), re-queues jobs from dead workers (`_onWorkerDead`), applies retry-with-backoff (`_retryOrFail`), falls back to local in-process builds when no worker is available (`_drainLocally`), and enforces the minimum-execution guarantee (`_enforceMinimumGuarantee`). It also owns the tenant list (`addTenant`, `reloadTenants`) and **dataset**

mode (`setDataset` , `_planDatasetJobs`) — when a dataset is loaded it splits the records across the apps. It emits a `state` event on every change.

`src/master/fault-manager.js` — The **Fault Tolerance System** (Requirement 11). It owns the worker registry and worker liveness. It detects dead workers two ways — missed heartbeats (a periodic sweep) and child-process exit — and emits `worker-dead` so the coordinator can re-queue lost work. `killWorker()` powers the dashboard's fault-simulation button.

`src/master/deployer.js` — The **Deployment & Launch System** (Requirement 10). `deployAndLaunch()` launches **every** generated app as an independent process on its own unique port and waits until each is reachable. To keep many launches fast, the first app is `npm install` -ed and the rest clone that `node_modules` (each app still gets its own copy). `shutdownAll()` stops them cleanly with the master.

`src/master/dashboard-server.js` — The master's single HTTP surface. It serves the dashboard UI, exposes the master API, runs the **reverse proxy** that maps `/appN` to each deployed app (with a `/appN → /appN/` redirect so relative asset URLs resolve), and runs the **WebSocket server** that streams state and logs to every connected dashboard.

`dashboard/` — The live monitoring UI (Requirement 12)

`dashboard/index.html` — The dashboard page structure: a top bar with the phase badge and control buttons, and seven panels (resources, decision, build progress, workers, deployments, fault events, live log).

`dashboard/styles.css` — A dark "control-room" theme: responsive CSS grid, colored status tags, progress bar, and log styling (decisions highlighted).

`dashboard/app.js` — The dashboard client. It opens a WebSocket to the master, auto-reconnects, and renders every panel from the streamed state. It wires the **Apps** count box, the **Upload Config** and **Upload Data** buttons, the dataset banner, **Start Generation**, **Refresh Resources**, and per-worker **Kill** buttons to the master API.

`tests/` — Unit tests (mirroring `src/`)

`tests/resource/decision-engine.test.js` — 9 tests: clamping, abundant resources, the MAX_APPS cap, memory as the binding constraint, high-load halving, the always-≥1 guarantee, reasoning output, and the explicit `appCount` override.

`tests/resource/resource-analyzer.test.js` — 2 tests: the simulated low-resource snapshot and a complete live snapshot.

`tests/jobs/job-queue.test.js` — 6 tests: registration & stats, FIFO order, drain detection, failed-job handling, retry reset, and disk persistence.

`tests/common/config-loader.test.js` — 6 tests: valid/invalid tenants, slug rules, loading all bundled configs, sort/uniqueness, and the system config.

`tests/engine/scaffolder.test.js` — 4 tests: placeholder rendering, generating a complete verifiable project, atomic regeneration, and unique ports.

`tests/common/chunker.test.js` — 6 tests: even splitting, remainder distribution, full coverage, never more chunks than records, empty input, and single-chunk mode.

Total: 33 unit tests, all passing.

`tools/` — Documentation tooling

`tools/generate-pdf.js` — Converts `DOCUMENTATION.md` to a styled `DOCUMENTATION.pdf` (renders the markdown to HTML, then prints it via headless Microsoft Edge).

Runtime-generated directories

`output/` — Where the generated independent applications are written (`output/app1` , `output/app2` , ...). Created at runtime.

`logs/` — `master.log` , one `worker-N.log` per worker, and the persisted `queue-state.json` . Created at runtime.

---

8. The Generated Application

When ADAGDS generates 2 apps, the output is:

```
/output
  /app1
  /app2
```

Each `appN` folder is a **fully independent, standalone project**:

```
output/app1/
  package.json          manifest — only dependency: express
  package-lock.json     created by `npm install`
  .env                  environment variables (PORT, APP_ID, …)
  README.md             per-app run instructions
  build.js              build-validation script
  src/
    server.js           Express server (static + /api/info + /health)
    config.js           loads this app's own config
  config/
    app.config.json     the tenant spec, baked in (isolated config)
  public/
    index.html          themed, configuration-driven UI
    styles.css          theme colors injected from the tenant
    app.js              front-end logic
```

It runs **on its own**, with no shared runtime dependency on any other app:

```
cd output/app1
npm install
npm start
```

---

9. Requirement-to-Implementation Map

#	Exam Requirement	Implemented by
1	Single base codebase	`engine/app-engine.js`, `scaffolder.js`, `engine/templates/`
2	Config-driven multi-app	`config/tenants/*.json`, `common/config-loader.js`
3	Distributed job execution	`master/*` + `worker/worker.js` (separate processes)
4	Parallel generation engine	`coordinator.js`, `jobs/job-queue.js`, `resource/scheduler.js`
5	**Independent codebase generation**	`engine/scaffolder.js` → `/output/appN`
6	Multi-tenant isolation	each app's own `.env`, `config/`, port — no shared state
7	Resource-aware scheduling	`resource/resource-analyzer.js`, `scheduler.js`
8	Adaptive workload decision	`resource/decision-engine.js`
9	Minimum execution guarantee	`coordinator.js` (`_enforceMinimumGuarantee`)
10	Deployment & launch	`master/deployer.js` + reverse proxy
11	Fault tolerance	`master/fault-manager.js`, persisted `job-queue.js`
12	Logging & monitoring	`common/logger.js`, `dashboard/`

---

10. Fault Tolerance — In Detail

ADAGDS survives worker failure without losing any job:

- **Two detection paths** — the Fault Manager declares a worker dead when it either misses several heartbeats (a periodic sweep) or its child process exits.
- **Job re-queue** — when a worker dies holding a job, the `worker-dead` event makes the coordinator return that job to the queue.
- **Retry with exponential backoff** — each job may be retried up to `MAX_RETRIES` (3) times; the job is held (not dropped) during the backoff.
- **Partial-build recovery** — the scaffolder writes to a `.tmp` directory and renames atomically; a leftover temp directory is cleaned and rebuilt.
- **No job loss** — every queue state change is persisted to `logs/queue-state.json`.
- **Local fallback executor** — if *every* worker is dead, the master builds the remaining jobs in-process, so the run still finishes.
- **Minimum execution guarantee** — even under the worst conditions, at least 1–2 applications are built and verified.

---

11. How to Run & Use

Prerequisites: Node.js 18 or newer.

Install (once):

```
npm install
```

Start the system:

```
npm start
```

Open the dashboard at **http://localhost:8080**. Then:

1. The dashboard immediately shows a live **Resource Analysis** snapshot.
2. Click **Start Generation**. Watch the adaptive decision, jobs distributed across workers in parallel, deployments, and the live log.
3. Generated apps appear in `/output` and are all auto-launched — each gets a clickable link at `http://localhost:8080/appN`.

Choosing how many apps: type a number in the **Apps** box on the dashboard before clicking **Start Generation**, or set `appCount` in `config/system.json`. Leave it blank (`"auto"`) to let the system decide. The count is still capped at 20 and reduced if resources are genuinely insufficient.

Uploading your own config: click **Upload Config** on the dashboard and pick a `.json` file shaped like those in `config/tenants/`. It is validated, saved, and available for the next run.

Dataset mode — split a dataset across apps: click **Upload Data** and pick a `.json` file that is an **array of records**. A banner confirms it loaded. Set a number in the **Apps** box, then **Start Generation** — ADAGDS slices the dataset evenly across that many independent, working data apps, each serving and displaying its own slice (`data/records.json`) with no database. **Clear dataset** in the banner returns to normal config-driven mode.

Fault-tolerance demo: click **Kill** on a worker — it is declared dead, its job re-queued, and the run still completes.

Low-resource demo: set `"forceLowResources": true` in `config/system.json` and restart — the decision engine reduces the app count, and the minimum guarantee still builds 1–2 apps.

Run a generated app independently:

```
cd output/app1
npm install
npm start
```

---

12. Testing

Tests use Node's built-in test runner (`node:test`) — no extra dependency. They live in `/tests` , mirroring the `/src` structure exactly.

```
npm test
```

Coverage: the decision engine, resource analyzer, job queue, config loader, and scaffolder — happy paths, edge cases, and failure scenarios.

Result: 33 tests, 33 passing.

---

13. Verification Evidence

The system was run end-to-end and verified:

Check	Result
Unit tests	33 / 33 passing
Master + worker nodes boot	6 workers registered
Adaptive decision logged	"Decided to generate 15 applications... binding constraint: maxByConfigs"
Parallel generation	15 / 15 apps built, 0 failures
Independent app runs standalone	`output/app5` → `npm install && npm start` → served correctly
Deployment + reverse proxy	`http://localhost:8080/app1` → Aurora Analytics
Fault detection	killed worker → declared dead, fault event recorded
Decision logging	6 `DECISION` -level entries in `logs/master.log`

ADAGDS — Adaptive Distributed Application Generation & Deployment System. Author: rafay9696@gmail.com